ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»

Параллельные вычисления

ЛАБОРАТОРНАЯ РАБОТА № 5

Тема: Использование векторных расширений системы команд процессора на примере решения задач обработки одномерных массивов

Группа: БПМ-22-ПО-1

Студент: Нургалиев Родион Дамирович

Вариант: 20

Ссылка на репозиторий с кодом:

https://github.com/LLrodyaLL/parallel_computing_misis_nurgaliev_r_d_bpm_22_po_1/tree/main/lab05

2026

Вариант 20

Тип данных: int8 | Операция: масштабирование | Расширение: SSE2 → SSE → SSE2

1. Цель работы

Ознакомиться с базовыми принципами векторизации вычислений в соответствии с SIMD-принципом обработки информации. Реализовать операцию поэлементного масштабирования одномерного массива значений типа **int8** тремя способами: на языке C++, с использованием скалярных SIMD-команд и с использованием векторных SIMD-команд расширения SSE2/SSE. Сравнить производительность реализаций при различных степенях раскрутки циклов.

2. Индивидуальное задание

В соответствии с вариантом 20 (таблица 12 методических указаний) задано следующее:

- Тип данных: **int8 (8-разрядное целое со знаком)**
- Операция: **масштабирование — $a'[i] = a[i] \cdot k, k \in \mathbb{R}$**
- Расширение: **SSE2 → SSE → SSE2** (целочисленный ввод/вывод через SSE2, вещественная арифметика через SSE)
- Размер массива: N = 1 000 000 элементов, k = 2.3
- Раскрутка цикла: 1, 2, 4 и 8 итераций

Математическая формула операции масштабирования:

$$a'[i] = a[i] \cdot k, \quad i = 0, 1, \ldots, N-1, \quad k \in \mathbb{R}, \quad a[i], a'[i] \in \text{int8}$$

При умножении 8-битных целых на вещественный коэффициент результат не помещается в диапазон int8, поэтому необходимо выполнять преобразование типов с округлением и насыщением (clamp) к границам [−128; 127].

3. Описание реализации

3.1. Структура проекта

Проект построен с использованием системы сборки CMake (C++17, компилятор MSVC). Исходные файлы:

- src/main.cpp - точка входа, замер времени, вывод результатов
- src/scaling.cpp - реализации трёх вариантов масштабирования
- include/scaling.h - объявления функций и типа AlignedInt8Vector
- include/aligned_allocator.h - STL-совместимый аллокатор с выравниванием по 16 байт
- include/timer.h - высокоточный таймер на std::chrono::high_resolution_clock

3.2. Выравнивание данных

Для эффективной работы инструкций загрузки/сохранения MOVAPS/MOVDQA необходимо выравнивание адреса массива по границе 16 байт. Реализован собственный STL-совместимый аллокатор `AlignedAllocator<T, 16>`, который использует `_aligned_malloc` (MSVC) или `posix_memalign` (GCC/Clang). Тип `AlignedInt8Vector` определён как `std::vector<int8_t, AlignedAllocator<int8_t, 16>>`.

3.3. Реализация 1 - C++ (эталон)

Скалярная реализация на языке C++. Каждый элемент массива умножается на коэффициент k в вещественном типе float, результат округляется с насыщением до диапазона int8:

```
int8_t clamp_round_to_int8(float value) {
    long rounded = std::lround(value);
    if (rounded < INT8_MIN) return INT8_MIN;
    if (rounded > INT8_MAX) return INT8_MAX;
    return static_cast<int8_t>(rounded);
}

// В цикле:
dst[i] = clamp_round_to_int8(static_cast<float>(src[i]) * k);
```

Функция поддерживает раскрутку цикла через параметр `unroll` с использованием конструкции `switch/case` с `[[fallthrough]]`.

3.4. Реализация 2 - SIMD скалярный (SSE2 → SSE → SSE2)

В данной реализации каждый элемент массива обрабатывается по отдельности, однако для выполнения операции используются SIMD-регистры. Процесс для одного элемента:

- int8 → int32: значение элемента передаётся в младший скаляр регистра XMM через `_mm_cvtsi32_ss`
- Умножение на k: скалярная SSE-инструкция `_mm_mul_ss` (MULSS)
- float → int32: `_mm_cvtss_si32` с банковским округлением (round-to-nearest)
- Ручное насыщение результата до диапазона int8 и запись в dst[i]

```
static inline int8_t simd_scalar_convert_one(int8_t value, float k) {
    __m128 vf = _mm_cvtsi32_ss(_mm_setzero_ps(), (int)value); // SSE2->SSE
    __m128 vk = _mm_set_ss(k);
    __m128 res = _mm_mul_ss(vf, vk);                          // SSE
    int out = _mm_cvtss_si32(res);                            // SSE->SSE2
    // ручное clamp:
    if (out < INT8_MIN) out = INT8_MIN;
    if (out > INT8_MAX) out = INT8_MAX;
    return (int8_t)out;
}
```

3.5. Реализация 3 - SIMD векторный (SSE2 → SSE → SSE2)

Ключевая реализация: обработка сразу 16 элементов (128 бит = 16 × int8) одним проходом через SIMD-конвейер. Вспомогательная функция `process_16_bytes_vector` выполняет следующие шаги:

- Загрузка 16 байт: `_mm_load_si128` (SSE2, выровненная загрузка)
- Расширение int8 → int16: знак-бит через `_mm_cmpgt_epi8(zero, bytes)`, затем `_mm_unpacklo_epi8 / _mm_unpackhi_epi8`
- Расширение int16 → int32: аналогично через `_mm_unpacklo/hi_epi16`, итого 4 регистра по 4 × int32
- Конвертация int32 → float: `_mm_cvtepi32_ps` × 4 (SSE2 → SSE)
- Векторное умножение на k: `_mm_mul_ps` × 4 (4 float за операцию, SSE)
- Конвертация float → int32: `_mm_cvtps_epi32` × 4 (SSE → SSE2)
- Упаковка int32 → int16: `_mm_packs_epi32` × 2 (с насыщением, SSE2)
- Упаковка int16 → int8: `_mm_packs_epi16` × 1 (с насыщением, SSE2)
- Сохранение 16 байт: `_mm_store_si128` (SSE2)

Раскрутка цикла в векторной реализации выполняется с шагом `16 × unroll` байт за итерацию, обеспечивая обработку до 128 элементов за проход (при unroll = 8).

4. Результаты измерений

Измерения выполнялись для массива из N = 1 000 000 элементов типа int8, k = 2.3. Каждая функция запускалась 10 раз, фиксировалось среднее время. Тестирование проводилось в конфигурациях Debug (/Od) и Release (/O2).

4.1. Конфигурация Debug

Таблица результатов - конфигурация Debug

Раскрутка	Реализация	Время (мс)	Ускорение vs C++
x1	C++ (эталон)	13.302	1.00x
	SIMD скалярный (SSE2→SSE→SSE2)	7.816	1.70x
	SIMD векторный (SSE2→SSE→SSE2)	1.912	6.96x
x2	C++ (эталон)	13.226	1.00x
	SIMD скалярный (SSE2→SSE→SSE2)	6.950	1.90x
	SIMD векторный (SSE2→SSE→SSE2)	1.816	7.28x
x4	C++ (эталон)	13.789	1.00x
	SIMD скалярный (SSE2→SSE→SSE2)	6.781	2.03x
	SIMD векторный (SSE2→SSE→SSE2)	1.793	7.69x
x8	C++ (эталон)	12.888	1.00x
	SIMD скалярный (SSE2→SSE→SSE2)	6.544	1.97x
	SIMD векторный (SSE2→SSE→SSE2)	1.807	7.13x

4.2. Конфигурация Release

Таблица результатов - конфигурация Release

Раскрутка	Реализация	Время (мс)	Ускорение vs C++
x1	C++ (эталон)	14.608	1.00x
	SIMD скалярный (SSE2→SSE→SSE2)	1.385	10.55x
	SIMD векторный (SSE2→SSE→SSE2)	0.166	88.00x
x2	C++ (эталон)	15.010	1.00x
	SIMD скалярный (SSE2→SSE→SSE2)	1.136	13.21x
	SIMD векторный (SSE2→SSE→SSE2)	0.163	92.09x
x4	C++ (эталон)	15.606	1.00x
	SIMD скалярный (SSE2→SSE→SSE2)	1.039	15.02x
	SIMD векторный (SSE2→SSE→SSE2)	0.156	100.04x
x8	C++ (эталон)	14.936	1.00x
	SIMD скалярный (SSE2→SSE→SSE2)	0.907	16.47x
	SIMD векторный (SSE2→SSE→SSE2)	0.162	92.20x

5. Анализ результатов и выводы

5.1. Сравнение скоростей

В конфигурации **Debug** (оптимизации компилятора отключены):

- Скалярный SIMD быстрее C++ в ~1.7–2.0 раза. Ускорение достигается за счёт того, что SSE-регистр загружает float и выполняет умножение за один такт, тогда как C++-вариант вынужден вызывать std::lround из libc.
- Векторный SIMD быстрее C++ в ~7 раз. Это объясняется одновременной обработкой 16 элементов одной XMM-операцией.

В конфигурации **Release** (оптимизация /O2):

- Скалярный SIMD быстрее C++ в ~11–16 раз. Компилятор с /O2 агрессивно раскрывает и конвейеризирует SIMD-код, тогда как C++-вариант ограничен вызовом std::lround.
- Векторный SIMD быстрее C++ в ~88–100 раз. Это крупнейший выигрыш: компилятор эффективно конвейеризирует векторные операции, а 16-кратный параллелизм данных полностью реализуется.

5.2. Влияние раскрутки цикла

В конфигурации **Debug** раскрутка даёт умеренный эффект: вектор ускоряется с 1.912 мс до 1.793 мс (раскрутка × 4) - прирост ~6%. В конфигурации **Release** раскрутка заметна для скалярного SIMD (1.385 → 0.907 мс, ~34%), но почти не влияет на векторный (0.166 → 0.156 мс, ~6%), так как компилятор уже сам агрессивно оптимизирует внутренний цикл.

5.3. О корректности результатов

Оба SIMD-варианта показывают «Нет» при сравнении с эталоном C++. Это не ошибка алгоритма, а **закономерное расхождение режимов округления**: функция `std::lround` реализует «round half away from zero» (арифметическое), а инструкция `_mm_cvtss_si32` / `CVTSS2SI` использует режим «round half to even» (банковское/четное). Для граничных значений (например, 2.5 × k) результаты отличаются на 1 единицу LSB. Если требуется полная битовая совместимость с C++-версией, необходимо заменить `_mm_cvtss_si32` на `_mm_cvt_roundss_si32` с флагом `_MM_FROUND_TO_NEG_INF` и ручным смещением.

5.4. Выводы

- Применение SIMD-расширений (SSE2/SSE) обеспечивает кратное ускорение обработки массивов даже без ручного вмешательства в оптимизацию: скалярный SIMD ускоряет в 2–16 раз, векторный — в 7–100 раз.
- Конфигурация Release демонстрирует существенно большее ускорение, чем Debug: в Release компилятор конвейеризирует SIMD-инструкции, устраняет накладные расходы вызовов и эффективнее планирует исполнение.
- Раскрутка цикла приносит наибольший выигрыш для скалярных SIMD-реализаций в Release; для векторных реализаций её вклад незначителен — узким местом становится пропускная способность памяти, а не накладные расходы цикла.
- Использование выровненных массивов (AlignedAllocator<T, 16>) и выровненных инструкций загрузки/хранения (_mm_load_si128 / _mm_store_si128) является необходимым условием корректной и эффективной работы SIMD-кода.
- При промышленном применении следует учитывать возможные расхождения режимов округления между операциями FPU/скалярного C++ и SIMD-инструкциями и явно выбирать нужный режим.